

# Prompt Engineering Strategies for LLM-based Qualitative Coding of Psychological Safety in Software Engineering Communities: A Controlled Empirical Study

Moaath Alshaikh  
Federal University of Bahia  
Salvador, Bahia, Brazil  
moaathalshaikh@ufba.br

Tasneem Alshaher  
Federal University of Bahia  
Salvador, Bahia, Brazil  
tasneemalshaher@ufba.br

Ricardo Vieira  
Federal University of Bahia  
Salvador, Bahia, Brazil  
revieira@ufba.br

Beatriz Santana  
Federal University of Bahia  
Salvador, Bahia, Brazil  
santanab@ufba.br

Clelio Xavier  
Federal University of Bahia  
Salvador, Bahia, Brazil  
clelio.xavier@ufba.br

Jose Amancio  
State University of Feira de Santana  
Feira de Santana, Bahia, Brazil  
zeamancio@uefs.br

Glaucio Carneiro  
Federal University of Sergipe  
Aracaju, Sergipe, Brazil  
glaucio.carneiro@dcomp.ufs.br

Julio Leite  
Federal University of Bahia  
Salvador, Bahia, Brazil  
julioleite.ufba@gmail.com

Savio Freire  
Federal Institute of Ceara  
Morada Nova, Ceará, Brazil  
savio.freire@ifce.edu.br

Manoel Mendonca  
Federal University of Bahia  
Salvador, Bahia, Brazil  
manoel.mendonca@ufba.br

## Abstract

Qualitative analysis plays a pivotal role in understanding the human and social aspects of software engineering. However, it remains a demanding process shaped by the subjective interpretation of individual researchers and sensitive to methodological choices such as prompt design. Recent advancements in Large Language Models (LLMs) offer promising opportunities to support this type of analysis, although their reliability in reproducing human qualitative reasoning under varying prompting conditions remains largely untested. This study presents a controlled empirical evaluation of three LLMs—Claude Haiku, DeepSeek-Chat, and Gemini 2.5 Flash—across two prompt engineering strategies (zero-shot and multi-shot closed coding), using Cohen’s  $\kappa$  as the primary agreement metric over ten independent runs per configuration. Results suggest that multi-shot prompting significantly improves agreement for Claude Haiku ( $\Delta\kappa = +0.034$ , Wilcoxon  $p = 0.004$ ) but not for DeepSeek-Chat or Gemini 2.5 Flash. Intra-model stability varies substantially—DeepSeek-Chat and Claude Haiku exhibit the lowest variance ( $SD \approx 0.017$ ), while Gemini 2.5 Flash is the least stable ( $SD = 0.038$ ). A systematic over-prediction of “Sharing Negative Feedback” is identified across all models (bias ratios up to 5.25 $\times$ ), alongside consistent under-prediction of “Expressing Concerns.” Collectively, these findings provide empirical evidence for prompt engineering guidelines in LLM-assisted qualitative coding for software engineering research.

## CCS Concepts

- **Software and its engineering**  $\rightarrow$  Empirical software validation;
- **Computing methodologies**  $\rightarrow$  Natural language processing.

## Keywords

Prompt Engineering, LLMs, Qualitative Coding, Psychological Safety, Software Engineering

*Preprint version of a paper accepted at the 1st International Workshop on Prompt Engineering for Software Engineering (PROMPT-SE 2026), co-located with the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026), Glasgow, Scotland, United Kingdom, June 9–12, 2026.*

## 1 Introduction

Large language models (LLMs) are among the most prominent recent developments in artificial intelligence, emerging as promising tools for qualitative text analysis and interpretation across academic disciplines [3]. In software engineering (SE), the growing interest in understanding human factors has intensified the adoption of qualitative methods to explore social phenomena such as psychological safety, team communication, and interpersonal conflict [12]. LLMs have been applied in this domain to analyze professional programming forums and support evidence-based reasoning [7], yet their reliability under varying prompting conditions remains largely untested.

Prompt engineering has emerged as a vital research area for ensuring reliable results from language models [9], yet current empirical evidence on how specific prompting strategies affect qualitative coding performance is limited. Questions of direct practical relevance—whether zero-shot prompting is sufficient, whether multi-shot examples improve agreement, whether models exhibit systematic category bias, and whether LLM outputs are stable across repeated runs—remain unanswered in the context of qualitative coding for SE.

This study addresses this gap through a controlled empirical evaluation of three state-of-the-art LLMs—Claude Haiku, DeepSeek-Chat, and Gemini 2.5 Flash—across two prompt configurations applied to a human-coded Gold Standard of 116 quotes related to psychological safety in SE communities on Stack Exchange. Each configuration was executed ten independent times per model to measure both mean agreement and intra-model stability—a dimension frequently overlooked in comparable studies. Four research questions guide this study:

- **RQ1:** To what extent can LLMs reproduce human qualitative coding of psychological safety statements in software engineering discussions, as measured by Cohen’s  $\kappa$  and per-class F1 scores?
- **RQ2:** How does prompt design (zero-shot vs. multi-shot) influence the reliability of LLM-based qualitative coding?
- **RQ3:** How stable are LLM qualitative coding results across repeated runs under the same prompt configuration?
- **RQ4:** Do LLMs exhibit systematic bias when coding psychological safety categories compared with human analysts?

## 2 Related Work

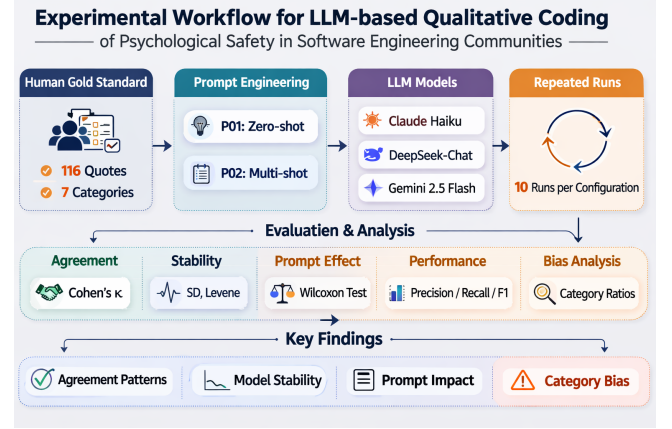
Barros et al. [3] conducted a systematic mapping study demonstrating that LLMs can substantially improve both efficiency and analytical accuracy in qualitative research. Within SE, Bano et al. [2] found persistent concerns around bias, transparency, and interpretive reliability in LLM-assisted qualitative analysis. Lenberg et al. [8] provided methodological guidelines emphasizing the balance between automation and human interpretation. Gao et al. [6] introduced CollabCoder, demonstrating how LLMs can strengthen analytical consistency in cooperative qualitative analysis. Leça et al. [7] described LLMs as a “new frontier” for empirical SE.

On the prompt engineering side, Liu et al. [9] provided a systematic survey of prompting methods including zero-shot and multi-shot configurations. Wei et al. [13] demonstrated that chain-of-thought prompting can elicit reasoning in LLMs. White et al. [14] proposed a catalog of prompt patterns for structured classification tasks. Chen et al. [4] examined prompt engineering in terms of fairness and reproducibility. Despite these advances, most existing studies evaluate LLM performance in a single run without assessing cross-run stability, and few compare multiple models under identical conditions on the same human-coded dataset—gaps this study directly addresses.

## 3 Research Method

### 3.1 Reference Dataset

The analysis relies on a human-coded reference dataset compiled by Santana et al. [10], including 116 quotes from the Stack Exchange Project Management (SEPM) and Software Engineering (SESE) communities. All quotes were manually classified into seven behavioral codes from Edmondson’s framework [5]: *Admitting Mistakes* (AM,  $n = 3$ ), *Disagreeing with Suggestions/Ideas* (DS,  $n = 35$ ), *Drawing Attention to Errors* (DAE,  $n = 5$ ), *Expressing Concerns* (EC,  $n = 50$ ), *Recommending Changes* (RC,  $n = 14$ ), *Seeking Help* (SH,  $n = 5$ ), and *Sharing Negative Feedback* (SNF,  $n = 4$ ). The label distribution is markedly imbalanced—EC accounts for 43.1% of all quotes—motivating the bias analysis in Section 5. Santana et al.



**Figure 1: LLM-based Exploratory Workflow for LLM-based Qualitative Coding of Psychological Safety in Software Engineering Communities**

### Algorithm 1 LLM-based Exploratory Workflow with Human Oversight

**Require:** 116 quotes; 7 codes [5]; Models = {Claude Haiku, DeepSeek-Chat, Gemini 2.5 Flash}; Prompts = {P01: zero-shot, P02: multi-shot}; GoldStandard [10]; Runs = 10

**Ensure:**  $\kappa$  scores, stability metrics, bias indices

- 1: **P01 – Zero-Shot:** classify 116 quotes  $\times$  3 models  $\times$  10 runs (0 example/label)
- 2: Compute  $\kappa$  (LLM<sub>r</sub>, GS); record per-class F1 scores; compute category bias ratios
- 3: **P02 – Multi-Shot:** repeat with 7 examples/label  $\times$  3 models  $\times$  10 runs
- 4: Wilcoxon( $\kappa_{P01}$ ,  $\kappa_{P02}$ ); Levene variance test; cross-model comparison

employed a multi-researcher coding process with consensus resolution of divergences; the authors report a 4.67% inter-researcher disagreement rate, though no formal inter-rater reliability statistic (e.g., Cohen’s  $\kappa$ ) is provided in the original study.

### 3.2 Methodological Workflow

Figure 1 illustrates the overall methodological workflow, which proceeds through four stages: (1) dataset preparation and gold standard establishment, (2) prompt engineering design, (3) automated LLM coding with repeated runs, and (4) statistical evaluation. Algorithm 1 provides procedural pseudocode for each configuration across all three models and ten independent runs.

## 4 Experimental Design and Setup

### 4.1 Prompt Engineering Design

Prompt engineering served as both an analytical control and a bias-mitigation mechanism [1, 9, 11]. Each prompt follows a standardized schema: role assignment, psychological safety definition, step-by-step instructions, code definitions, illustrative examples, and a structured output format (ID–Category–Challenge–Quote). The key design differences are: **P01** provides 0 example per code (zero-shot baseline); **P02** provides 7 examples per code (multi-shot enrichment). Complete prompt texts are provided in Appendix A. Table 1 summarizes the two configurations evaluated. P01 provides no annotated examples—only category names and decision rules—and therefore constitutes a true zero-shot configuration. Any earlier

**Table 1: Prompt Configurations Used in Closed-Coding Phases**

| ID  | Phase      | Key Design Choice                                                                     | Scope                    |
|-----|------------|---------------------------------------------------------------------------------------|--------------------------|
| P01 | Zero-Shot  | No annotated examples; category names & classification rules only; 7 labels available | All 3 models; 116 quotes |
| P02 | Multi-Shot | 7 examples/code; 7 labels available                                                   | All 3 models; 116 quotes |

reference to “1 example/code” described a definitional clause within the category description, not an annotated training example.

## 4.2 Experimental Setup

We evaluated: Claude Haiku (claude-haiku-4-5-20251001), DeepSeek-Chat (deepseek-chat, accessed via `api.deepseek.com`, OpenAI-compatible endpoint), and Gemini 2.5 Flash (gemini-2.5-flash). Each was evaluated on both P01 and P02 over ten independent runs, yielding 60 classification runs in total ( $3 \times 2 \times 10$ ). Repeated runs are necessary because LLM outputs may vary across executions due to stochastic sampling processes. All three models were queried with `temperature = 0` to minimise stochastic variation; no seed parameter was applied (the parameter is unsupported by the Gemini API and was not set for Claude Haiku or DeepSeek-Chat). No `top_p` or `frequency_penalty` overrides were applied beyond API defaults.

## 4.3 Evaluation Metrics

**Cohen’s  $\kappa$  (Agreement):** Per-run agreement corrected for chance; aggregated as mean  $\pm$  SD across ten runs. Interpreted using the Landis–Koch scale:  $< 0.20$  Slight,  $0.20$ – $0.40$  Fair,  $0.40$ – $0.60$  Moderate,  $0.60$ – $0.80$  Substantial.

**Intra-Model Stability (SD of  $\kappa$ ):** Variance of  $\kappa$  across ten independent runs. Levene’s test compares variance between models at each configuration.

**Bias Ratio:** Ratio of LLM-predicted count for a given category to its human-coded Gold Standard count. Ratio = 1.00 indicates no bias; values  $> 1.20$  flag over-prediction; values  $< 0.80$  flag under-prediction. Note that the Bias Ratio captures distributional volume—the aggregate predicted count versus the gold-standard count—and does not reflect per-instance classification accuracy. It should therefore be interpreted alongside per-class F1 scores rather than in isolation.

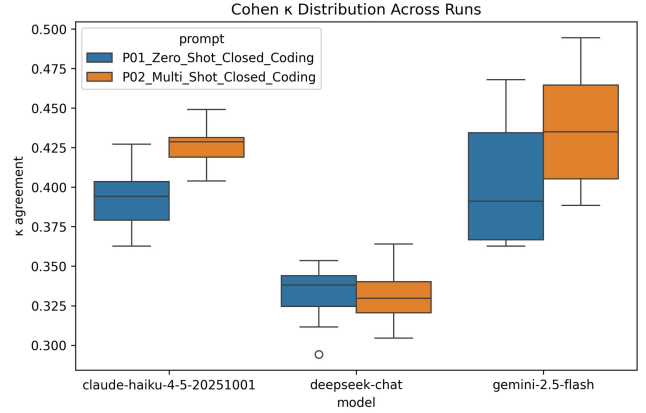
Per-class F1 scores (majority vote, ten runs) and Wilcoxon signed-rank test (P01 vs. P02,  $n = 10$  pairs, with Cohen’s  $d$  as effect size) complement the primary metrics.

## 5 Results

### 5.1 RQ1 – Agreement Between LLM Coding and Human Coding

Figure 2 shows the  $\kappa$  distribution across ten runs for all models under both prompt configurations. Table 2 summarizes mean  $\kappa$ , SD, and range.

Overall, the results suggest fair to moderate agreement between LLM-generated labels and the human gold standard. Under P01,  $\kappa$  values ranged from 0.332 (DeepSeek-Chat) to 0.403 (Gemini 2.5



**Figure 2: Cohen’s  $\kappa$  distribution across ten independent runs for all three models under P01 (zero-shot) and P02 (multi-shot). Boxes show interquartile range; whiskers extend to  $1.5 \times \text{IQR}$ ; circles denote outliers.**

**Table 2: Cohen’s  $\kappa$  Results — All Configurations ( $N = 116$ , 10 runs)**

| Prompt | Model            | $\bar{\kappa}$ | SD    | Min   | Max   | Interp.  |
|--------|------------------|----------------|-------|-------|-------|----------|
| P01    | Claude Haiku     | 0.392          | 0.018 | 0.363 | 0.427 | Fair     |
| P01    | DeepSeek-Chat    | 0.332          | 0.017 | 0.294 | 0.354 | Fair     |
| P01    | Gemini 2.5 Flash | 0.403          | 0.038 | 0.363 | 0.468 | Fair     |
| P02    | Claude Haiku     | <b>0.426</b>   | 0.011 | 0.404 | 0.449 | Moderate |
| P02    | DeepSeek-Chat    | 0.331          | 0.017 | 0.304 | 0.364 | Fair     |
| P02    | Gemini 2.5 Flash | 0.437          | 0.035 | 0.388 | 0.495 | Moderate |

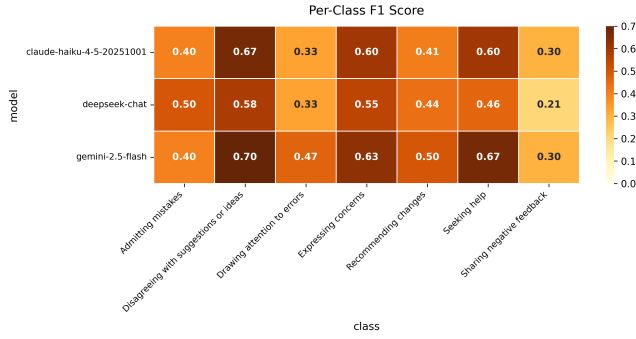
Flash), all within the Fair range. Under P02, Claude Haiku and Gemini 2.5 Flash crossed into the Moderate range ( $\kappa = 0.426$  and  $0.437$  respectively), while DeepSeek-Chat remained essentially unchanged ( $\kappa = 0.331$ ).

Per-class F1 scores are illustrated in Figure 3. Across all models, “Disagreeing with Suggestions or Ideas” achieves the highest F1 scores ( $0.58$ – $0.70$ ), while minority categories such as “Sharing Negative Feedback” ( $F1 = 0.21$ – $0.30$ ) and “Admitting Mistakes” ( $F1 \approx 0.33$ – $0.40$ ) are systematically under-predicted. These patterns confirm that LLMs can partially reproduce human coding of common categories but struggle with semantically rare or overlapping ones. Caution is warranted when interpreting F1 scores for minority categories (AM:  $n = 3$ ; SNF:  $n = 4$ ; DAE:  $n = 5$ ): with support sizes this small, a single misclassification shifts F1 by  $0.10$ – $0.33$  points, rendering these values indicative rather than statistically reliable.

### 5.2 RQ2 – Effect of Prompt Engineering

Table 3 reports Wilcoxon signed-rank test results comparing  $\kappa$  distributions between P01 and P02 for each model.

Claude Haiku showed a statistically significant improvement from P01 to P02 ( $\Delta\kappa = +0.034$ ,  $p = 0.004$ ,  $d = 2.41$ ), crossing the Landis–Koch boundary from Fair to Moderate. The large Cohen’s  $d$  ( $2.41$ ) reflects near-perfect directional consistency across all ten runs rather than the magnitude of the mean difference;  $d$  computed on small paired samples ( $n = 10$ ) is sensitive to low within-pair



**Figure 3: Per-class F1 scores (majority vote, 10 runs) aggregated across both prompt configurations. Color scale: green = high F1, red = low. “Disagreeing with Suggestions or Ideas” achieves the highest F1; “Sharing Negative Feedback” is consistently under-predicted. F1 scores for minority categories (AM:  $n = 3$ ; SNF:  $n = 4$ ; DAE:  $n = 5$ ) should be treated as indicative only due to very small support sizes.**

**Table 3: Wilcoxon Signed-Rank Test — P01 vs. P02 per Model (RQ2)**

| Model            | $\tilde{\kappa}_{P01}$ | $\tilde{\kappa}_{P02}$ | $\Delta\kappa$ | W    | $p$          | $d$   |
|------------------|------------------------|------------------------|----------------|------|--------------|-------|
| Claude Haiku     | 0.392                  | 0.426                  | +0.034         | 1.0  | <b>0.004</b> | 2.41  |
| DeepSeek-Chat    | 0.332                  | 0.331                  | -0.001         | 25.0 | 0.846        | -0.02 |
| Gemini 2.5 Flash | 0.403                  | 0.437                  | +0.033         | 10.0 | 0.084        | 0.89  |

$n = 10$  pairs per test.

**Table 4: Intra-Model Stability — SD of  $\kappa$  Across 10 Runs (RQ3)**

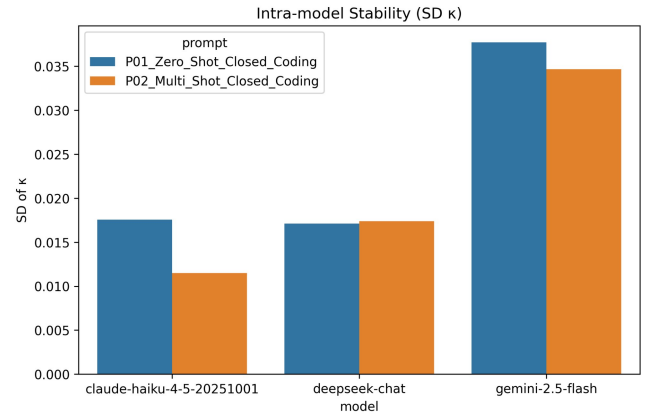
| Model            | $SD_{P01}$ | $SD_{P02}$   | Direction                   |
|------------------|------------|--------------|-----------------------------|
| Claude Haiku     | 0.018      | <b>0.011</b> | -0.007 (improved)           |
| DeepSeek-Chat    | 0.017      | 0.017        | $\approx 0$ (unchanged)     |
| Gemini 2.5 Flash | 0.038      | 0.035        | -0.003 (slight improvement) |

variance, and when all ten P02 runs consistently exceed their P01 counterparts by a small but stable margin,  $d$  inflates relative to the mean difference—a known behaviour of effect-size estimates on small paired samples. This large effect size suggests that multi-shot prompting meaningfully benefits models that are already in the Fair range. Gemini 2.5 Flash showed a similar positive trend ( $\Delta\kappa = +0.033$ ,  $p = 0.084$ ), marginally missing the  $p < 0.05$  threshold. DeepSeek-Chat showed virtually no change ( $\Delta\kappa = -0.001$ ,  $p = 0.846$ ), suggesting its coding behavior is largely insensitive to example count in this closed-coding task.

### 5.3 RQ3 – Stability of LLM Coding Across Runs

Figure 4 and Table 4 report the standard deviation of  $\kappa$  across ten runs per configuration.

A stability gradient emerges across models. Claude Haiku and DeepSeek-Chat exhibit the lowest variance under P01 ( $SD = 0.018$  and  $0.017$  respectively), indicating comparatively reproducible coding behavior. Gemini 2.5 Flash shows the highest instability ( $SD =$



**Figure 4: Intra-model stability: SD of  $\kappa$  across 10 runs per configuration. Lower values indicate more reproducible coding behavior. Multi-shot prompting reduces variance for Claude Haiku.**

**Table 5: “Expressing Concerns” Bias Ratios Across Configurations**

| Model            | P01 Count | P01 Ratio | P02 Count | P02 Ratio |
|------------------|-----------|-----------|-----------|-----------|
| Human GS         | 50        | 1.00      | 50        | 1.00      |
| Claude Haiku     | 37        | 0.74      | 41        | 0.82      |
| DeepSeek-Chat    | 35        | 0.70      | 43        | 0.86      |
| Gemini 2.5 Flash | 32        | 0.64      | 37        | 0.74      |

0.038), with individual run  $\kappa$  values spanning a range of 0.105 (0.363–0.468). Multi-shot prompting reduces variance for Claude Haiku ( $SD : 0.018 \rightarrow 0.011$ ), while DeepSeek-Chat and Gemini exhibit marginal changes.

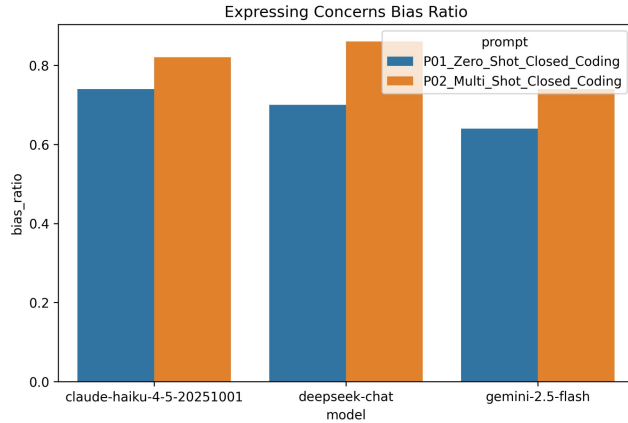
These findings reinforce that reporting a single-run agreement score is methodologically insufficient. Repeated executions should be considered a minimum standard when evaluating LLM-based qualitative coding systems. Although the SD of  $\kappa$  is modest for Claude Haiku and DeepSeek-Chat ( $\approx 0.017$ ), Gemini 2.5 Flash’s individual run  $\kappa$  spans a range of 0.105 (0.363–0.468)—a difference that crosses the Fair–Moderate boundary and would materially affect interpretation if only a single run were reported. Furthermore, ten independent runs constitute the minimum sample size required for a valid Wilcoxon Signed-Rank Test ( $n \geq 10$ ), which underpins the inferential analysis in RQ2.

### 5.4 RQ4 – Bias in Psychological Safety Coding

Figure 5 and Table 5 present bias ratios for the “Expressing Concerns” category across models and prompt configurations. Table 6 provides a full breakdown of bias ratios across all seven categories.

The bias analysis reveals a multi-directional pattern. Most notably, “Expressing Concerns”—the dominant category (43.1%)—is *under*-predicted by all three models under P01 (ratios: 0.74, 0.70, and 0.64). Multi-shot prompting partially corrects this, raising ratios to 0.82, 0.86, and 0.74 respectively, but EC remains under-predicted across all configurations.





**Figure 5: “Expressing Concerns” prediction bias ratio across models under P01 (zero-shot) and P02 (multi-shot). Values below 1.0 indicate under-prediction relative to the Gold Standard ( $n = 50$ , 43.1%). All three models under-predict this category.**

**Table 6: Full Category Bias Ratios Under P01 (Zero-Shot)**

| Model            | EC   | DS   | RC   | SH   | DAE  | SNF  | AM   |
|------------------|------|------|------|------|------|------|------|
| Human GS         | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| Claude Haiku     | 0.74 | 0.69 | 1.14 | 1.20 | 2.40 | 4.75 | 0.67 |
| DeepSeek-Chat    | 0.70 | 0.57 | 1.29 | 2.80 | 1.80 | 4.75 | 0.33 |
| Gemini 2.5 Flash | 0.64 | 0.74 | 0.93 | 2.20 | 2.40 | 5.25 | 0.33 |

*EC=Expressing Concerns, DS=Disagreeing/Suggestions, RC=Recommending Changes, SH=Seeking Help, DAE=Drawing Attention to Errors, SNF=Sharing Negative Feedback, AM=Admitting Mistakes.*

The most severe over-prediction bias is found in “Sharing Negative Feedback” (SNF), with ratios of 4.75× (Claude Haiku and DeepSeek-Chat) and 5.25× (Gemini 2.5 Flash) under P01. “Drawing Attention to Errors” (DAE) is similarly over-predicted (1.80×–2.40×). This pattern suggests all three models conflate semantically adjacent categories—EC, SNF, and DAE all involve challenging or critical communicative acts—by over-labeling statements as SNF or DAE rather than EC. Under P02, SNF ratios remain elevated (4.50×, 3.25×, 5.25×), confirming that this is a model-level semantic prior rather than a prompt-level artifact. “Admitting Mistakes” and “Disagreeing with Suggestions” are consistently under-predicted across all configurations.

## 5.5 Summary of Findings

Across all experiments, four key results emerge. First, all three models achieve fair to moderate agreement with human qualitative coding ( $\kappa = 0.33$ – $0.44$ ), confirming partial but limited reproducibility. Second, multi-shot prompting significantly improves agreement for Claude Haiku ( $p = 0.004$ ) and shows a positive trend for Gemini 2.5 Flash, while leaving DeepSeek-Chat unaffected. Third, Gemini 2.5 Flash exhibits the highest run-to-run variance, reinforcing the need for repeated-run evaluation. Fourth, systematic bias is present in all models: “Sharing Negative Feedback” is severely over-predicted (up to 5.25×), while “Expressing Concerns” is consistently

under-predicted—a pattern that persists across both prompt configurations.

## 6 Discussion

**RQ1 – LLMs Achieve Partial but Limited Agreement.** The fair to moderate  $\kappa$  scores (0.33–0.44) confirm that LLMs can partially reproduce human qualitative coding of psychological safety statements, but a substantial proportion of disagreements persist. Per-class F1 analysis reveals that performance is not uniform: models perform best on frequent and semantically distinct categories (“Disagreeing with Suggestions or Ideas”) and worst on rare or overlapping ones (“Sharing Negative Feedback”, “Admitting Mistakes”). These findings align with prior work showing that LLMs can approximate human annotation in well-defined tasks [2], while confirming their reliability is insufficient for fully autonomous qualitative analysis.

**RQ2 – Prompt Design Differentially Affects Models.** The significant improvement for Claude Haiku ( $p = 0.004$ ,  $d = 2.41$ ) suggests that some models benefit considerably from in-context examples, while others—like DeepSeek-Chat—are unaffected. This challenges the assumption that multi-shot prompting offers uniform benefits [9]. The practical implication is that prompt strategy should be calibrated per model rather than applied uniformly, and researchers should empirically evaluate prompt sensitivity before adopting a fixed strategy.

**RQ3 – Stability Warrants Multi-Run Reporting Standards.** Gemini 2.5 Flash’s higher variance ( $SD = 0.038$ ) versus Claude Haiku and DeepSeek-Chat ( $SD \approx 0.017$ ) highlights a key methodological concern: a single-run evaluation may yield a misleading picture of reliability. The stability improvement for Claude Haiku under multi-shot prompting ( $SD : 0.018 \rightarrow 0.011$ ) demonstrates that prompt design affects not only mean agreement but also consistency. These findings strengthen the case for standardized multi-run evaluation protocols in LLM-based qualitative coding research.

**RQ4 – Bias Is Pervasive and Category-Specific.** The systematic over-prediction of SNF and DAE alongside under-prediction of EC suggests a shared semantic tendency across models to interpret ambiguous interpersonal statements as expressions of criticism. The EC under-prediction pattern—where models distribute EC-appropriate quotes across SNF and DAE—may reflect semantic overlap among the three categories, all of which involve challenging or negative communicative acts. That these biases persist across both prompt configurations confirms they reflect model-level priors rather than prompt-level artifacts. Bias monitoring across all categories—not only the dominant one—should therefore be a standard diagnostic step in LLM-assisted qualitative coding. It bears noting that the Bias Ratio measures distributional volume rather than per-instance accuracy; the category-level F1 scores reported in Section 5 provide the complementary instance-level perspective.

**Actionable Guidelines.** Four actionable guidelines emerge from this study: (1) prompt strategy should be calibrated per model; multi-shot prompting offers significant benefits for some models but not others; (2) multi-run evaluation should be adopted as a minimum standard; (3) full category-level bias monitoring should be embedded in any LLM coding workflow with imbalanced label distributions; (4) minority category performance requires special

attention, as macro-averaged metrics may obscure near-zero F1 on rare but theoretically important categories.

## 7 Threats to Validity

**Construct:** Seven behavioral codes may not capture all nuances of psychological safety. All categories derive from Edmondson’s validated framework [5]. Agreement metrics (Cohen’s  $\kappa$ , macro-F1) may not fully capture the nuanced reasoning involved in qualitative interpretation. Semantic overlap among EC, SNF, and DAE—all involving critical or negative communicative acts—may independently contribute to the observed bias patterns, irrespective of model limitations. Disentangling model-level from scheme-level sources of classification error remains an open challenge and is reflected in the category-level discriminative guidance embedded in the prompts.

**Internal:** Prompt wording is a primary validity threat, mitigated by a fixed schema across all configurations. The P01 single-example-per-code design may introduce implicit labeling bias; the specific example selected per category could influence model priors.

**External:** 116 quotes from two Stack Exchange communities may not represent industrial, organizational, or cross-cultural settings. The generalizability of the observed bias patterns to other qualitative coding frameworks with different label distributions remains an open empirical question.

**Reliability:** Ten independent runs explicitly address non-determinism; however, ten runs may still underestimate variance for highly stochastic configurations. The stability results should be interpreted as approximate estimates of run-to-run variability rather than precise variance characterizations. Although  $\text{temperature} = 0$  is near-deterministic in practice, it does not guarantee fully identical outputs across runs; the 10-run protocol was adopted precisely to capture residual non-determinism.

**Statistical power:** Wilcoxon tests on ten-pair comparisons have moderate power. The significant result for Claude Haiku is robust; the near-significant result for Gemini ( $p = 0.084$ ) should be interpreted with caution and warrants replication with larger run counts.

**Gold Standard reliability:** The Gold Standard from Santana et al. [10] was produced through a multi-researcher consensus process (4.67% disagreement rate reported); however, no formal inter-rater reliability statistic (e.g., Cohen’s  $\kappa$ ) is reported in the original study. This limits our ability to contextualise the LLM–human agreement scores against a human–human baseline.

## 8 Conclusion

This study presented a controlled empirical evaluation of three LLMs across two prompt engineering configurations applied to a human-coded Gold Standard of 116 psychological safety quotes from Stack Exchange SE communities. Running each model–prompt pair ten independent times provides empirical grounding for prompt engineering guidelines in LLM-assisted qualitative coding for SE. Key findings: **(1)** all three models achieve fair to moderate agreement with human coding ( $\kappa = 0.33\text{--}0.44$ ); **(2)** multi-shot prompting significantly improves agreement for Claude Haiku ( $\Delta\kappa = +0.034$ ,  $p = 0.004$ ), while having no meaningful effect on DeepSeek-Chat;

**(3)** Gemini 2.5 Flash exhibits the highest run-to-run variance, highlighting the need for multi-run evaluation protocols; **(4)** systematic over-prediction of “Sharing Negative Feedback” (up to  $5.25\times$ ) and under-prediction of “Expressing Concerns” persist across all models and configurations, reflecting model-level semantic priors. These findings motivate investigation of chain-of-thought and contrastive prompting strategies, standardized multi-run reporting requirements, and extension to larger and more diverse datasets. Zero-shot and multi-shot configurations were selected as they represent the minimal and enriched baselines most directly comparable in the closed-coding literature; chain-of-thought and contrastive prompting are explicitly deferred to future work to maintain a focused experimental comparison. An analysis of inter-LLM agreement—examining whether models converge on the same labels independently of the gold standard—also represents a valuable complementary direction for future investigation.

## Availability of Artifacts

The replication package associated with this study is publicly archived on Zenodo and includes the reference datasets, prompt specifications, analysis pipeline, and execution scripts used in the experiments.

Moaath Alshaikh. (2026). *PROMPT-SE 2026 Replication Package (v1.0.0)*. Zenodo. <https://doi.org/10.5281/zenodo.20032899>

The corresponding GitHub repository is available at: <https://github.com/moaathalshaikh/PROMPT-SE-2026-Replication-Package>

## References

- [1] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, et al. 2023. A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.
- [2] Muneera Bano, Rashina Hoda, Didar Zowghi, and Christoph Treude. 2024. Large language models for qualitative research in software engineering: exploring opportunities and challenges. *Automated Software Engineering* 31, 1 (2024), 8.
- [3] Cauã Ferreira Barros, Bruna Borges Azevedo, Valdemar Vicente Graciano Neto, Mohamad Kassab, Marcos Kalinowski, Hugo Alexandre D. Do Nascimento, and Michelle C.G.S.P. Bandeira. 2025. Large Language Model for Qualitative Research: A Systematic Mapping Study. In *Proc. of the IEEE/ACM Int. Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE)*. IEEE, 48–55.
- [4] Banghao Chen, Zhaofeng Zhang, Nicolas Langréné, and Shengxin Zhu. 2025. Unleashing the potential of prompt engineering for large language models. *Patterns* 6, 6 (2025), 101260.
- [5] Amy Edmondson. 1999. Psychological safety and learning behavior in work teams. *Administrative Science Quarterly* 44, 2 (1999), 350–383.
- [6] Jie Gao, Yuchen Guo, Gionnieve Lim, Tianqin Zhang, Zheng Zhang, Toby Jia-Jun Li, and Simon Tangi Perrault. 2024. CollabCoder: a lower-barrier, rigorous workflow for inductive collaborative qualitative analysis with large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–29.
- [7] Matheus De Morais Leça, Lucas Valença, Reydney Santos, and Ronnie De Souza Santos. 2025. Applications and implications of large language models in qualitative analysis: A new frontier for empirical software engineering. In *Proc. of the IEEE/ACM Int. Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE)*. IEEE, 36–43.
- [8] Per Lenberg, Robert Feldt, Lucas Gren, Lars Göran Wallgren Tengberg, Inga Tidefors, and Daniel Graziotin. 2024. Qualitative software engineering research: Reflections and guidelines. *Journal of Software: Evolution and Process* 36, 6 (2024), e2607.
- [9] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *Comput. Surveys* 55, 9, Article 195 (2023), 35 pages.
- [10] Beatriz Silva De Santana, Sávio Freire, Leandro Cruz, Lidivânio Monte, Manoel Mendonça, and José Amancio Macedo Santos. 2023. Exploring psychological safety in software engineering: Insights from stack exchange. In *Proceedings of the XXXVII Brazilian Symposium on Software Engineering (SBES)*. 503–513.
- [11] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, et al. 2024. The prompt report: A systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608* (2024).
- [12] Carolyn B. Seaman, Rashina Hoda, and Robert Feldt. 2025. Qualitative Research Methods in Software Engineering: Past, Present, and Future. *IEEE Transactions on Software Engineering* 51, 3 (2025), 783–788.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [14] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP)*. Article 5.

# A Full Prompt Specifications

## A.1 P01: Prompt 01 (Zero-Shot)

You are a software engineering researcher specializing in qualitative analysis of psychological safety in software development teams.

Your task is to perform qualitative coding of the quotes provided below by identifying and classifying psychological safety challenges described in each quote.

The goal is to identify challenges related to psychological safety, such as interpersonal risk-taking, learning anxiety, defensive behaviors, or resistance to change.

=== CONTEXT FOR CLASSIFICATION ===

Psychological safety refers to a person's perception that they can take interpersonal risks in a team environment without fear of negative consequences such as embarrassment, rejection, or punishment.

=== UNIT OF ANALYSIS ===

Each quote represents a single unit of analysis.

Even if multiple psychological challenges appear in the same quote, identify and classify only the ONE primary challenge that best represents the main issue expressed in the quote.

=== INSTRUCTIONS ===

Step 1 - Read and Analyze the Quote  
Carefully read each quote and analyze the psychological challenge described.

Step 2 - Classify the Challenge  
Assign the quote to ONE of the following categories.

=== CATEGORIES ===

- Expressing concerns
- Recommending changes
- Sharing negative feedback
- Disagreeing with suggestions or ideas
- Drawing attention to errors
- Seeking help
- Admitting mistakes

=== CATEGORY SELECTION RULE ===

Each quote must be assigned to ONE primary category only.

If multiple categories seem applicable, choose the category that best represents the central psychological challenge described in the quote.

The category value must match exactly one of the seven category names listed above.

=== AMBIGUITY HANDLING RULE ===

If a quote does not clearly match any category, select the closest category based only on explicit evidence in the quote.

If no explicit signal is present, choose the category supported by the strongest textual evidence.

Avoid speculative interpretation of meaning, intent, or context not stated in the quote.

=== DECISION PRIORITY RULE ===

If a quote explicitly mentions a mistake, bug, defect, or failure, prioritize the category "Drawing attention to errors" over "Expressing concerns" or "Sharing negative feedback".

=== EVIDENCE RULE ===

The identified challenge must be grounded strictly in the content of the quote. Do not infer motivations, intentions, or contextual information that are not explicitly stated in the quote.

=== CHALLENGE DESCRIPTION RULE ===

The field ``challenge\\_identified`` must contain a concise description between 5 and 15 words summarizing the psychological challenge.

Prefer neutral, descriptive wording rather than interpretive language.

=== COVERAGE RULE ===

Analyze every quote exactly once.

=== OUTPUT FORMAT (STRICT) ===

Return ONLY valid JSON.

The output must be a JSON array where each element contains the following fields:

- \* id\\_quote
- \* category
- \* challenge\\_identified
- \* related\\_quote

Return the results ONLY as a JSON array.

## A.2 P02: Prompt 02 (Multi-Shot)

You are a software engineering researcher specializing in qualitative analysis of psychological safety in software development teams.

Your task is to perform qualitative coding of the quotes provided below by identifying and classifying psychological safety challenges described in each quote.

The goal is to identify challenges related to psychological safety, such as interpersonal risk-taking, learning anxiety, defensive behaviors, or resistance to change.

=== CONTEXT FOR CLASSIFICATION ===

Psychological safety refers to a person's perception that they can take interpersonal risks in a team environment without fear of negative consequences such as embarrassment, rejection, or punishment.

=== UNIT OF ANALYSIS ===

Each quote represents a single unit of analysis.

Even if multiple psychological challenges appear in the same quote, identify and classify only the ONE primary challenge that best represents the main issue expressed in the quote.

=== INSTRUCTIONS ===

Step 1 - Read and Analyze the Quote  
Carefully read each quote and analyze the psychological challenge described.

Step 2 - Classify the Challenge  
Assign the quote to ONE of the following categories.

=== CATEGORIES ===

- Expressing concerns
- Recommending changes
- Sharing negative feedback
- Disagreeing with suggestions or ideas
- Drawing attention to errors
- Seeking help
- Admitting mistakes

=== ANNOTATED EXAMPLES ===

Example 01  
id\\_quote: Quote12  
category: Disagreeing with suggestions or ideas  
challenge\\_identified: Team members hesitate to raise concerns about effort estimation in Scrum



Prompt Engineering Strategies for LLM-based Qualitative Coding  
of Psychological Safety in Software Engineering Communities:  
A Controlled Empirical Study

Example 02

id\\_quote: Quote16  
category: Drawing attention to errors  
challenge\\_identified: Recurring performance issues in Front-End team  
despite extra effort

Example 03

id\\_quote: Quote01  
category: Expressing concerns  
challenge\\_identified: Difficulties dealing with argumentative and  
uncooperative team members

Example 04

id\\_quote: Quote02  
category: Recommending changes  
challenge\\_identified: Encouraging team to adopt agile vertical slicing  
practices

Example 05

id\\_quote: Quote55  
category: Seeking help  
challenge\\_identified: Difficulties obtaining support from a reluctant  
experienced engineer

Example 06

id\\_quote: Quote106  
category: Sharing negative feedback  
challenge\\_identified: Receiving harsh criticism during a first code review

Example 07

id\\_quote: Quote42  
category: Admitting mistakes  
challenge\\_identified: Struggles with delegation and task management as  
a project manager

=== OUTPUT FORMAT (STRICT) ===

Return ONLY valid JSON.

The output must be a JSON array where each element contains the  
following fields:

- \* id\\_quote
- \* category
- \* challenge\\_identified
- \* related\\_quote

Return the results ONLY as a JSON array.